OBJECT ORIENTED PROGRAMMING WITH C++

ASSIGNMENT 3 – CO3

Constructors, Destructors and Operator Overloading

Student details	Fill before submission
Name	HARIPRASAD R
Register Number	192421179
Submission Date	12-11-2026
GitHub Repository	https://github.com/<username>/DSA01-<RegisterNumber>-Assignment3

Important execution and evidence note

This guide contains worked designs, compilable examples, output predictions and explanations. You must compile and run every task yourself, capture genuine terminal screenshots, record your compiler version and timings, and commit each task separately. Do not claim sanitizer reports or benchmark values unless you actually obtained them.

```
g++ --version
g++ -std=c++17 -Wall -Wextra -pedantic task.cpp -o task
# Windows PowerShell:
.\task.exe
```

Use -fsanitize=address -g for sanitizer builds where your compiler supports AddressSanitizer. On Windows, support depends on the MinGW distribution; if unavailable, use a supported GCC/Clang environment or Valgrind on Linux and record the environment.

Q1 – Constructors and Constructor Overloading (6 Marks)

1. Book class design

```
#include <iostream>
#include <string>
#include <stdexcept>
using namespace std;

class Book {
    string title;
    string author;
    int copies;

    static int validCopies(int n) {
        if (n < 0) throw invalid_argument("Copies cannot be negative");
        return n;
    }
public:
    Book() : Book("Untitled", "Unknown", 0) {} // default, delegates
    explicit Book(const string& t) : Book(t, "Unknown", 1) {} // donated
    Book(const string& t, const string& a, int n)
        : title(t), author(a), copies(validCopies(n)) {}

    void print() const {
        cout << title << " | " << author << " | copies=" << copies << '\n';
    }
};

int main() {
    Book empty;
    Book donated("C++ Primer");
    Book full("Clean Code", "Robert Martin", 4);
    empty.print(); donated.print(); full.print();
```

```
    try { Book invalid("Bad Record", "Nobody", -2); }
    catch (const invalid_argument& e) { cout << "Error: " << e.what() << '\n'; }
}
```

All constructors use member initializer lists. The default constructor delegates to the full constructor, and the title-only constructor delegates as well. The validation function rejects negative copies before the object can be constructed.

2. Constructor rules and object states

- The compiler supplies a default constructor only when no user-declared constructor prevents it. Once a parameterised constructor is declared, the compiler does not automatically add a no-argument constructor.
- A member initializer list directly initializes members. It is required for const members and references, avoids default-construction followed by assignment, and is generally more efficient.
- Members are initialized in the order they are declared in the class, not the order written in the initializer list.
- Object states after construction: empty = (Untitled, Unknown, 0); donated = (C++ Primer, Unknown, 1); full = (Clean Code, Robert Martin, 4).

3. Default constructor removal, order warning and explicit

```
// If Book() is removed, this fails:
Book shelf[3]; // no matching zero-argument constructor for each element

// Demonstration of wrong initializer-list order:
class WrongOrder {
    int first;
    int second;
public:
    WrongOrder() : second(20), first(second) {} // -Wreorder and unsafe read
};

// explicit prevents implicit conversion:
class Label {
public:
    explicit Label(const std::string&) {}
};
// Label x = std::string("Novel"); // error with explicit
// Label x(std::string("Novel"));  // valid direct initialization
```

Compile the wrong-order example with `-Wall -Wextra -Wreorder`. It may warn that second is listed before first even though first is initialized first. Do not rely on the value read from second while first is being initialized.

4. Alternative: one constructor with defaults

```
Book(string t = "Untitled", string a = "Unknown", int n = 0)
    : title(t), author(a), copies(validCopies(n)) {}
```

A single constructor with default arguments can represent all three cases, but overloads communicate the supported creation situations more clearly. Both designs perform O(1) member initialization for fixed-size members; storing strings may allocate memory proportional to their lengths. Centralizing validation in one full constructor (and delegating to it) reduces duplicated checks.

Design choice: retain overloaded constructors and delegation because the three creation cases have distinct meanings and share one validation path. Mark a converting single-argument constructor explicit when implicit conversion would be surprising or unsafe.

Q2 – Copy Constructors: Shallow and Deep Copies (7 Marks)

1. Deep-copy Rule-of-Three implementation

```
#include <iostream>
#include <string>
using namespace std;

class Prescription {
    int n;
    string* medicines;
public:
```

```
    explicit Prescription(int count)
        : n(count), medicines(count > 0 ? new string[count] : nullptr) {}

    Prescription(const Prescription& other)
        : n(other.n), medicines(n > 0 ? new string[n] : nullptr) {
        for (int i = 0; i < n; ++i) medicines[i] = other.medicines[i];
        cout << "[copy constructor] ";
    }

    Prescription& operator=(const Prescription& other) {
        if (this != &other) {
            string* fresh = other.n > 0 ? new string[other.n] : nullptr;
            for (int i = 0; i < other.n; ++i) fresh[i] = other.medicines[i];
            delete[] medicines;
            medicines = fresh;
            n = other.n;
        }
        return *this;
    }

    ~Prescription() { delete[] medicines; }

    void set(int i, const string& m) {
        if (i >= 0 && i < n) medicines[i] = m;
    }
    string get(int i) const {
        return (i >= 0 && i < n) ? medicines[i] : "";
    }
};

int main() {
    Prescription p(2);
    p.set(0,"Paracetamol"); p.set(1,"ORS");
    Prescription q = p;
    q.set(1,"Cetirizine");
    Prescription r(1);
    r = p;
    p.set(0,"Ibuprofen");
    cout << p.get(0) << ' ' << p.get(1) << " | "
         << q.get(0) << ' ' << q.get(1) << " | "
         << r.get(0) << ' ' << r.get(1) << '\n';
    r = r; // self-assignment is safely ignored
}
```

Expected visible result (the copy-constructor message appears before the final line):

```
[copy constructor] Ibuprofen ORS | Paracetamol Cetirizine | Paracetamol ORS
```

The required data output is exactly: `Ibuprofen ORS | Paracetamol Cetirizine | Paracetamol ORS`. The extra copy message is included because the implementation prints it to demonstrate when the copy constructor runs.

2. Shallow versus deep memory diagrams

```
SHALLOW COPY (unsafe raw pointer copy)
p.medicines ----+----> [ Paracetamol | ORS ]
q.medicines ----+       same allocation
Both destructors call delete[] on the same address -> double delete.

DEEP COPY
p.medicines ----------> [ Paracetamol | ORS ]
q.medicines ----------> [ Paracetamol | ORS ]  (separate allocation)
Each object owns and deletes its own array.
```

The compiler-generated copy constructor copies the pointer value, not the pointed-to array. Both objects then believe they own the same memory. Modifying one changes what the other sees, and destruction can cause double-free/use-after-free errors.

3. Rule of Three, copy constructor calls and measurements

- Rule of Three: if a class needs a user-defined destructor, copy constructor or copy assignment operator because it manages a resource, it usually needs all three.
- Copy constructor is used when a new object is initialized from another object, when an object is passed by value, and when an object is returned by value in cases where copy elision does not remove the copy.

- Passing by const reference does not copy the Prescription object. Passing by value may invoke a copy; compiler optimizations and C++17 guaranteed copy elision can affect observed counts in particular expressions.

Medicine count n	Expected deep-copy time	Extra array space
10	O(n)	O(n)
1,000	O(n)	O(n)
100,000	O(n)	O(n)

Measure your own times with std::chrono and report medians. A shallow pointer copy is O(1) but incorrect for independent ownership. The std::vector<string> alternative follows the Rule of Zero: its compiler-generated copy copies the vector elements safely. Copy time and storage remain O(n).

Sanitizer evidence: compile with `g++ -std=c++17 -g -fsanitize=address -fno-omit-frame-pointer q2.cpp -o q2`. Run the deep-copy version and record no sanitizer error. For the shallow version, create two owning objects from one another and let both destruct; record the actual double-free diagnostic. Do not invent sanitizer output; exact text varies by runtime.

Q3 – Dynamic Constructors and Destructors (7 Marks)

1. Dynamic two-dimensional seat map

```
#include <iostream>
using namespace std;

class Event {
    int rows, seats, booked;
    bool** map;
public:
    Event(int r, int s) : rows(r), seats(s), booked(0), map(nullptr) {
        if (rows > 0 && seats > 0) {
            map = new bool*[rows];
            int made = 0;
            try {
                for (; made < rows; ++made) {
                    map[made] = new bool[seats]{};
                }
            } catch (...) {
                while (made > 0) delete[] map[--made];
                delete[] map;
                throw;
            }
        }
        cout << "Create " << rows << "x" << seats << '\n';
    }

    bool book(int r, int s) {
        if (!map || r < 0 || r >= rows || s < 0 || s >= seats || map[r][s])
            return false;
        map[r][s] = true; ++booked; return true;
    }
    bool cancel(int r, int s) {
        if (!map || r < 0 || r >= rows || s < 0 || s >= seats || !map[r][s])
            return false;
        map[r][s] = false; --booked; return true;
    }
    ~Event() {
        if (map) {
            for (int i = 0; i < rows; ++i) delete[] map[i];
            delete[] map;
        }
        cout << "Close event; booked=" << booked << '\n';
    }
};

int main() {
    Event stackEvent(2,3);
    stackEvent.book(0,1);
    {
```

```
        Event blockEvent(1,2);
        blockEvent.book(0,0);
    } // blockEvent destroyed here
    Event* heapEvent = new Event(3,4);
    heapEvent->book(2,2);
    delete heapEvent; // destructor called here
    Event zero(0,5);
}
```

This example uses a dynamic constructor: it allocates storage based on runtime dimensions. The destructor releases every row and then the row-pointer array. The constructor includes cleanup if allocation of a later row throws.

2. Lifetime and memory analysis

- Local stack objects are destroyed automatically in reverse order of construction when their scope ends.
- An object in an inner block is destroyed at the closing brace of that block.
- An object created with new remains alive until delete is called; delete invokes its destructor.
- For an array allocated with new T[n], use delete[]; using delete instead is undefined behaviour. Forgetting delete leaks the object and its allocations.
- A destructor has no parameters and no return type. It cannot be overloaded because destruction has one fixed role and is invoked automatically.
- RAII ties resource release to object lifetime, so normal scope exit and exception unwinding call the destructor automatically.

```
Row-pointer layout:
map ---> [row pointer 0] ---> [bool bool bool ...]
       [row pointer 1] ---> [bool bool bool ...]
       [row pointer 2] ---> [bool bool bool ...]
Allocations: one pointer array + one allocation per row = rows + 1.
```

For R rows and S seats, seat storage is R×S bool elements plus R row pointers (approximately R×S bytes for bool plus R×sizeof(bool*) bytes, excluding allocator overhead). For 10×20, 100×200 and 1,000×2,000, the logical seat storage is approximately 200 B, 20,000 B and 2,000,000 B respectively if sizeof(bool)=1; confirm sizeof(bool) on your compiler.

3. Predict the Logger output

Prediction before execution:

```
+A +B -B +AcopyAc use -Ac +P -P end -A
```

Explanation: a is constructed first. b is constructed in the inner block and destroyed at its closing brace. use(a) takes its argument by value, so a copy named Ac is constructed; use prints `use`, then the parameter copy is destroyed at function exit. The heap Logger P is constructed by new and destroyed by delete. `end` is printed, then local a is destroyed at main's closing brace.

The spaces are exactly those emitted by the stream operations; there is no newline in the supplied program.

4. Alternative layouts

```
// One contiguous allocation:
bool* seats = new bool[rows * cols]{};
bool occupied = seats[r * cols + c];
delete[] seats;

// Standard container alternative:
std::vector<std::vector<bool>> seats(
    rows, std::vector<bool>(cols, false));
```

All access methods are O(1). A contiguous block needs one allocation and no row-pointer array, improving locality. The row-pointer approach uses rows+1 allocations and R pointers but permits independently sized rows. vector<vector<bool>> is convenient and automatically manages memory, but vector<bool> is a specialized packed representation; vector<vector<char>> is often easier when ordinary element references are needed.

Q4 – Overloading Arithmetic and Stream Operators (7 Marks)

1. Money class and completed TODOs

```
#include <iomanip>
```

```
#include <iostream>
using namespace std;

class Money {
    long long paise;
public:
    Money(long long rupees = 0, int p = 0) : paise(rupees * 100 + p) {}

    Money operator+(const Money& other) const {
        Money result;
        result.paise = paise + other.paise;
        return result;
    }
    bool operator==(const Money& other) const {
        return paise == other.paise;
    }
    friend ostream& operator<<(ostream& out, const Money& m) {
        long long whole = m.paise / 100;
        long long fraction = m.paise % 100;
        if (fraction < 0) fraction = -fraction;
        out << "Rs " << whole << '.' << setw(2) << setfill('0') << fraction;
        return out;
    }
};

int main() {
    Money a(120,75), b(79,25);
    Money c = a + b;
    cout << c << '\n' << boolalpha << (c == Money(200)) << '\n';
    Money x(0,75), y(0,50);
    cout << x + y << '\n';
}
```

Expected output:

```
Rs 200.00
true
Rs 1.25
```

TODO 1 is the member operator+ returning a new Money. TODO 2 consists of the member operator== and friend stream insertion operator<<.

2. Operator design and arithmetic rules

- + returns a new value and leaves its operands unchanged. += conventionally modifies the left operand and returns Money& (*this) so chained updates work.
- << must be non-member because the left operand is std::ostream, not Money. A friend declaration allows it to access private paise.
- `a + b` is interpreted as `a.operator+(b)` for a member overload, or `operator+(a,b)` for a non-member overload.
- Subtraction that becomes negative must have a documented policy: allow signed balances, reject it, or throw an exception. This sample stores signed paise and can represent a negative result.
- Multiplying Money by Money is usually not meaningful because the unit becomes currency squared. Multiplying Money by a scalar quantity or rate can be meaningful with explicit rounding rules.

3. Alternative non-member and mixed scalar operands

```
friend Money operator+(const Money& a, const Money& b) {
    Money result;
    result.paise = a.paise + b.paise;
    return result;
}
// To support 100 + m, use a non-member overload:
friend Money operator+(long long rupees, const Money& m) {
    Money result(rupees);
    result.paise += m.paise;
    return result;
}
```

Member and non-member arithmetic are both O(1) time and O(1) extra space for this fixed-size representation. A non-member overload can allow implicit conversion on the left operand, such as 100 + m, whereas a member function on Money cannot be called on a built-in left operand. A named add() function has the same arithmetic complexity but may be more explicit.

Q5 – Overloading Increment, Comparison and Subscript Operators (6 Marks)

1. Time and Playlist implementation

```
#include <iostream>
#include <stdexcept>
#include <string>
#include <vector>
using namespace std;

class Time {
    int total;
public:
    Time(int h=0, int m=0, int s=0) : total(h*3600 + m*60 + s) {}
    Time& operator++() { ++total; return *this; } // prefix
    Time operator++(int) { Time old=*this; ++total; return old; } // postfix
    bool operator==(const Time& b) const { return total == b.total; }
    bool operator<(const Time& b) const { return total < b.total; }
    friend ostream& operator<<(ostream& o, const Time& t) {
        int h=t.total/3600, m=(t.total/60)%60, s=t.total%60;
        return o << h << ':' << (m<10?"0":"") << m << ':' << (s<10?"0":"") << s;
    }
};

class Playlist {
    vector<string> tracks;
public:
    Playlist(initializer_list<string> x) : tracks(x) {}
    string& operator[](int i) {
        if (i < 0 || i >= static_cast<int>(tracks.size()))
            throw out_of_range("Track index out of range");
        return tracks[i];
    }
    const string& operator[](int i) const {
        if (i < 0 || i >= static_cast<int>(tracks.size()))
            throw out_of_range("Track index out of range");
        return tracks[i];
    }
    int size() const { return static_cast<int>(tracks.size()); }
};

int main() {
    Time t(0,59,59);
    Time a = t++;
    Time b = ++t;
    cout << a << ' ' << b << ' ' << t << '\n';
    Playlist list{"Song A","Song B"};
    cout << list[0] << '\n';
    try { cout << list[list.size()] << '\n'; }
    catch (const out_of_range& e) { cout << e.what() << '\n'; }
}
```

Prefix returns the updated object by reference. Postfix has a dummy int parameter to distinguish it and returns the old value by copy. In the example, starting at 00:59:59: a receives 00:59:59, then prefix advances the object to 01:00:01 and b receives 01:00:01.

2. Given Predict the Output

Prediction before execution:

```
5 7 7
```

c starts at 5. `a=c++` copies the old value 5 into a and increments c to 6. `b=++c` increments c to 7 and returns the updated object, so b=7 and c=7. Postfix makes the extra copy; prefix does not need to create an old-value copy.

3. Subscript safety and complexity

- operator[] needs a non-const overload returning T& so callers can modify an element, and a const overload returning const T& for read-only objects.
- The sample throws std::out_of_range for negative indices and indices greater than or equal to size. Another valid policy is to return an error/status, but silently accessing invalid memory is unsafe.

- An at(i) method also performs a bounds check and throws. Built-in-style unchecked [] is faster by a small constant but can cause undefined behaviour for invalid indices.

All accesses are O(1); checked access adds a comparison and branch. Measure 10,000,000 valid accesses with the same data and checksum for checked and unchecked versions, at -O0 and -O2. Record medians; do not assume a measurable difference because optimization and timing noise may dominate.

Benchmark	-O0 median	-O2 median
Prefix ++, 10,000,000 calls		
Postfix ++, 10,000,000 calls		
Checked [] access		
Unchecked access		

Many programmers prefer ++i for class types because it updates and returns the object without preserving an old copy. For simple built-in integers, compilers usually optimize both forms equally.

Q6 – The Rules of Operator Overloading (7 Marks)

1. Text class using dynamic memory

```
#include <iostream>
#include <cstring>
#include <stdexcept>
using namespace std;

class Text {
    char* data;
public:
    Text(const char* s = "") {
        size_t n = strlen(s);
        data = new char[n+1];
        memcpy(data, s, n+1);
    }
    Text(const Text& other) {
        size_t n = strlen(other.data);
        data = new char[n+1];
        memcpy(data, other.data, n+1);
    }
    Text& operator=(const Text& other) {
        if (this != &other) {
            size_t n = strlen(other.data);
            char* fresh = new char[n+1];
            memcpy(fresh, other.data, n+1);
            delete[] data;
            data = fresh;
        }
        return *this;
    }
    ~Text() { delete[] data; }

    Text operator+(const Text& b) const {
        size_t n1=strlen(data), n2=strlen(b.data);
        char* temp=new char[n1+n2+1];
        memcpy(temp,data,n1);
        memcpy(temp+n1,b.data,n2+1);
        Text result(temp);
        delete[] temp;
        return result;
    }
    bool operator==(const Text& b) const { return strcmp(data,b.data)==0; }
    bool operator!=(const Text& b) const { return !(*this==b); }
    friend ostream& operator<<(ostream& o,const Text& t) { return o << t.data; }
};

int main() {
    Text a("Hello "), b("World"), c("");
    Text d = a + b;
    cout << d << '\n' << boolalpha << (d != c) << '\n';
```

```
    d = d; // self-assignment safe
}
```

The class uses deep copy and guards self-assignment. The temporary buffer is released after constructing the result. For production code, std::string is strongly preferred; it handles allocation, copying, exceptions and many edge cases.

2. Operators that cannot be overloaded and core rules

Operator	Reason
::	Scope resolution is part of name lookup and parsing syntax.
.	Member access has built-in language semantics.
.*	Pointer-to-member access syntax.
?:	Conditional operator has language-level evaluation rules.
sizeof	Compile-time size operator.

- An overloaded operator cannot change its precedence, associativity or number of operands (arity).
- At least one operand must be a class or enumeration type; operators cannot be redefined solely for built-in types.
- operator= must be a non-static member function.
- A friend is useful for symmetric operations or when the left operand is a different type, such as ostream << Text.

3. Copies and concatenation complexity

For `a + b + c`, evaluation is left associative: `(a+b)+c`. The first result contains n1+n2 characters; the second contains n1+n2+n3. A straightforward implementation copies O(2n1+n2+n3) characters across the two concatenations, which is O(n1+n2+n3), and creates intermediate temporary storage. The exact number of allocations depends on the implementation and copy elision.

The alternative `Text operator+(Text a, const Text& b) { a += b; return a; }` can reuse the by-value left operand's buffer if += is implemented with capacity management, but this class would need capacity tracking and careful reallocation. std::string is usually the best practical solution. Benchmark 100,000 concatenations using the same input sizes, reserve capacity where appropriate, and report actual medians.

AddressSanitizer should be used to detect leaks, double deletes and out-of-bounds writes. Test empty strings, long strings, copy construction, assignment, self-assignment and chained concatenation.

Q7 – Design a Complex-Number Class from First Principles (10 Marks)

Stage 1 – Requirements and data

- Represent impedance as real and imaginary double components: Z = a + jb.
- Support default construction (0+j0), real-only construction (R+j0), and real/imaginary construction.
- Support addition, subtraction, multiplication, division, equality, rectangular output and magnitude.
- Division by zero impedance must be detected and reported safely.

Stage 2 – Class design and formulas

```
Complex
  private: double re, im
  constructors: Complex(), Complex(real), Complex(real, imag)
  operators: + - * / == <<
  magnitude(): sqrt(re*re + im*im)

(a+jb) + (c+jd) = (a+c) + j(b+d)
(a+jb) - (c+jd) = (a-c) + j(b-d)
(a+jb) * (c+jd) = (ac-bd) + j(ad+bc)
(a+jb) / (c+jd) = ((ac+bd) + j(bc-ad)) / (c*c+d*d)
```

Stage 3 – Implementation and circuit verification

```
#include <cmath>
#include <iostream>
#include <stdexcept>
using namespace std;
```

```
class Complex {
    double re, im;
public:
    Complex() : re(0), im(0) {}
    Complex(double real) : re(real), im(0) {}
    Complex(double real, double imag) : re(real), im(imag) {}

    double real() const { return re; }
    double imag() const { return im; }
    double magnitude() const { return sqrt(re*re + im*im); }

    Complex operator+(const Complex& z) const {
        return Complex(re+z.re, im+z.im);
    }
    Complex operator-(const Complex& z) const {
        return Complex(re-z.re, im-z.im);
    }
    Complex operator*(const Complex& z) const {
        return Complex(re*z.re-im*z.im, re*z.im+im*z.re);
    }
    Complex operator/(const Complex& z) const {
        double den=z.re*z.re+z.im*z.im;
        if (den == 0.0) throw domain_error("Division by zero impedance");
        return Complex((re*z.re+im*z.im)/den,
                       (im*z.re-re*z.im)/den);
    }
    bool operator==(const Complex& z) const {
        const double eps=1e-9;
        return fabs(re-z.re)<eps && fabs(im-z.im)<eps;
    }
    friend ostream& operator<<(ostream& out,const Complex& z) {
        out << z.re << (z.im < 0 ? " - j" : " + j") << fabs(z.im);
        return out;
    }
};

int main() {
    try {
        const double pi=acos(-1.0), f=50.0, w=2*pi*f;
        const double R=10.0, L=0.1, C=100e-6;
        Complex Zr(R,0);
        Complex Zl(0,w*L);
        Complex Zc(0,-1.0/(w*C));

        Complex series=Zr+Zl+Zc;
        Complex parallel=(Zr*Zl)/(Zr+Zl);

        cout << "Series RLC impedance: " << series
             << "\nMagnitude: " << series.magnitude() << '\n';
        cout << "Parallel R-L impedance: " << parallel
             << "\nMagnitude: " << parallel.magnitude() << '\n';

        try {
            Complex zero;
            Complex invalid=Zr/zero;
            cout << invalid << '\n';
        } catch (const domain_error& e) {
            cout << "Error: " << e.what() << '\n';
        }
    } catch (const exception& e) {
        cerr << e.what() << '\n';
    }
}
```

The code calculates a series combination of R, L and C, and a parallel R-L combination using (Z1×Z2)/(Z1+Z2). It explicitly tests division by zero. The equality operator uses a tolerance because floating-point calculations are approximate.

Hand-check guidance

At f=50 Hz, ω=2πf. For R=10 Ω, L=0.1 H and C=100 µF: ZR=10+j0; ZL=jωL; ZC=−j/(ωC). The series impedance is 10+j(ωL−1/(ωC)). Calculate ω, XL and XC with a calculator and compare with the program to an appropriate decimal tolerance. For parallel R-L, calculate ZR×ZL and divide by ZR+ZL using the complex formulas above.

Stage 4 – Evaluation and std::complex alternative

std::complex<double> already implements arithmetic, magnitude and stream support, and is generally more thoroughly tested. A custom Complex class is useful for learning, domain-specific validation, formatting or additional impedance rules. Both have O(1) time per arithmetic operation and O(1) storage (two doubles). The custom class must explicitly handle division by zero and floating-point tolerance; std::complex follows the standard library's floating-point behavior, so an application may still need its own zero-denominator policy.

Remaining limitation: this implementation uses double precision and exact `den == 0.0` for division-by-zero detection. Very small denominators, numerical conditioning and unit consistency need more robust engineering treatment in a production circuit simulator.

Final GitHub and Evidence Checklist

- ☐Create repository named DSA01-<RegisterNumber>-Assignment3 and add faculty access if private.
- ☐Create Task01 through Task07 folders, each with source code and relevant input/output evidence.
- ☐Add README.md with your name, register number, compiler/version, exact compile commands and run instructions.
- ☐Compile every program with warnings enabled and fix unintended warnings.
- ☐Capture the Q1 missing-default-constructor error and -Wreorder warning.
- ☐Run Q2 shallow and deep versions under AddressSanitizer or Valgrind and preserve actual reports.
- ☐Run Q3 with correct deletion and with a deliberately missing delete only in a separate test; record the leak report if the tool detects it.
- ☐Record measured Q2 copy times, Q3 memory estimates, Q5 increment/access timings and Q6 std::string comparison timings.
- ☐Write all output predictions before execution and preserve those predictions in the report.
- ☐Make at least one meaningful progressive commit per task.
- ☐Put the working GitHub repository URL on the cover page and in the submission portal.

README.md template

```
# DSA01-<RegisterNumber>-Assignment3

- Name: <Your name>
- Register Number: <Your register number>
- Compiler: <g++ version>
- Language standard: C++17

## Compile
g++ -std=c++17 -Wall -Wextra -pedantic main.cpp -o main

## Run
Windows PowerShell: .\main.exe
Linux/macOS: ./main

## Tasks
Task01 – Constructors and overloading
Task02 – Deep copy and Rule of Three
Task03 – Dynamic constructor and destructor
Task04 – Money operator overloading
Task05 – Increment, comparison and subscript
Task06 – Text class and operator rules
Task07 – Complex-number class

For each task, record tests, predicted output, actual output,
compiler warnings, sanitizer results and measurements.
```

Final reminder: enter your own name/register number, repository URL, execution screenshots, sanitizer output and measured timings. The document provides example solutions, but the assignment requires you to execute, understand and defend your own work.